

Raspberry Pi 上の Embedded Xinu のマルチコア化

ワーカープール型 SMP の設計と実測

何が速くなり、何が速くならないか

児玉靖司

法政大学経営学部

yass@hosei.ac.jp

2026 年 7 月 16 日

Abstract

Raspberry Pi 上でベアメタル動作する Embedded Xinu を、搭載された 4 つの CPU コアを使うように拡張した。設計は、コア 0 が既存の単一コア OS（スケジューラ・アクター・ネットワーク・ウィンドウシステム）を一切変更せずに実行し続け、残るコアを純計算専用のワーカーとして起こす「ワーカープール型」である。対称型 SMP を採らないのは、既存ランタイムが非リエントラントであり、共有 ready キューが実績のある機能を破壊するためである。

本レポートの主眼は、速くなったという報告ではなく、**何が速くなり、何が速くならないかを実測で切り分けた点**にある。計算律速の処理は最大 $\times 3.99$ （ほぼ理想値）まで向上した一方、メモリ書込律速の処理は $\times 0.95$ – $\times 1.35$ に留まり、並列化の意味がないことが分かった。この結果は「ウィンドウシステムを 4 コアで速くできるか」という問いに否と答える根拠となった。またワーカープールの起動コストは実測 7–11 μs であり、これを下回る仕事は並列化するとかえって遅くなる（実測 $\times 0.38$ ）。この 2 つの境界が、並列化すべき対象を決める。

さらに、この「メモリ律速はスケールしない」という結論が **D-cache OFF** に固有であることを、D-cache を有効化する実験で実測した（第 6 節）。D-cache ON では、作業セットがキャッシュに収まる限りメモリ書込は理想的にスケールし（64 KB で $\times 4.01$ ）、超えると帯域律速へ回帰する（1 MB で $\times 1.40$ ）。ただし実フレームバッファはキャッシュ不可であり、かつ全画面描画はキャッシュを超えるため、ウィンドウシステムの結論自体は覆らない。D-cache の有効化は Cortex-A53 で set/way invalidate・SMPEN・DMA コヒーレンスを要し、とりわけ USB ドライバがスタック上のバッファを DMA する構造が最大の障害であることを明らかにした。

Raspberry Pi 3 についての実装と測定を第 5 節、D-cache 実験を第 6 節に示す。Raspberry Pi 4 および 5 についても同一の枠組みで追記予定である。

Contents

1 はじめに	3
2 背景と関連研究	3
2.1 既存研究：XinuPi3（Marquette、教育目的の対称型 SMP）	3
2.2 本研究との違い	3
3 共通設計：ワーカープール型 SMP	4
3.1 なぜ対称型 SMP を採らないか	4
3.2 ロックフリー調停が成立する理由	4
3.3 ジョブ引数は呼び出し側スタックに置く	4
3.4 プールの排他とフォールバック	5
4 測定方法	5
4.1 時計：割り込みではなくフリーランニングカウンタを読む	5
4.2 正しさの同時検証	5
4.3 絶対時間ではなく比を報告する	5

5	Raspberry Pi 3 (BCM2837)	5
5.1	対象と前提	5
5.2	コア起動：Pi 3 固有の差分	6
5.3	実測 1：計算律速の処理は 4 コアでスケールする	6
5.4	実測 2：メモリ律速の処理はスケールしない	7
5.5	実測 3：コア数スケーリング（Amdahl 曲線）	7
5.6	実測 4：プールの往復コストと損益分岐点	8
5.7	適用判断 1：ウィンドウシステムは並列化しない	8
5.8	適用判断 2：AIPL アクターは 1 箇所だけが該当する	8
5.9	発見した不具合	9
5.10	発見した測定バグ：不公平な基準が $\times 4.42$ を生んだ	10
5.11	副次的な発見：volatile グローバルの代償	10
6	実験：D-cache を有効化する（Level 2）	10
6.1	動機：支配変数はキャッシュ構成である	10
6.2	D-cache ON は「フラグを 1 にする」ではない	10
6.3	最大の障害：USB DMA はページ属性で直せない	11
6.4	実験の切り分け：USB を解かずに問いに答える	11
6.5	測定ハーネス	12
6.6	実測結果	12
6.7	付録：実機シリアル出力（D-cache ON）	13
6.8	本番化の試みと限界：USB DMA は越えられなかった	13
7	ボード横断比較	15
8	設計空間 GA：予測 vs 実機実測	15
8.1	狙い：GA の予測を実測と突き合わせる	15
8.2	結果：worker_pool が champion（実測が裏付ける）	15
8.3	一致とアーティファクト	16
8.4	最大の発見：支配変数が schema に無い	16
8.5	結論：GA は方向を、実測は真実を語る	16
9	結論	16

1 はじめに

Raspberry Pi に搭載される BCM2837 / BCM2711 / BCM2712 は、いずれも 4 コアの ARM プロセッサを持つ。しかし Embedded Xinu は単一コア OS であり、これらのボード上では 1 コアしか使わず、残る 3 コアは起動されないまま放置されていた。本作業の目的は、この遊休コアを実際の計算に使うことである。

本レポートは次の順で述べる。第 3 節で全ボード共通の設計方針を、第 4 節で測定方法を述べる。第 5 節以降がボード別の実装と実測である。第 7 節で横断比較を行い、第 9 節で結論を述べる。

2 背景と関連研究

Xinu は Douglas Comer が教科書『Operating System Design: The Xinu Approach』のために設計した、単一プロセッサ前提の軽量教育用 OS である。その ARM / MIPS 系組込み移植 **Embedded Xinu** は Marquette 大学の Dennis Brylow のグループが主導し、複数大学の OS 教育で使われている。本作業もこの Embedded Xinu の Raspberry Pi 3 移植を出発点とする。

2.1 既存研究：XinuPi3 (Marquette、教育目的の対称型 SMP)

Raspberry Pi 3 の 4 コア化そのものは、Xinu の本家グループが既に発表している。**XinuPi3**¹ は Embedded Xinu を Pi 3 へ移植し、**対称型マルチプロセッシング (full SMP)** を実現した。その主眼は**教育**にあり——「マルチコア概念を、真に並行なハードウェアで初学者に教える」ため——コアの起動、キャッシュコヒーレンシ、**コア間相互排他 (ロック)**、マルチコアスケジューリングを扱う。論文自身が「ベアメタルの教育用 OS でマルチコアを支えるものは他に無い」と述べ、その**存在と教材としての価値**を貢献としている。同グループは続けて、Pi 3 を用いた OS 教育・アセンブリでの並列計算教育の実践も報告している²。

2.2 本研究との違い

本レポートの研究は、XinuPi3 と**目的・方式・貢献**のいずれも異なる。

Table 1: XinuPi3 (既存) と本研究の違い

	XinuPi3 (Marquette)	本研究
目的	教育——マルチコア概念を教える教材	測定駆動の工学——稼働中の機能豊富な OS の並列性能を、退行させずに最大化
SMP 方式	対称型 (full SMP)	非対称ワーカプール型。コア 0 は既存 OS を無改変で走らせ、コア 1-3 を計算ワーカーに
キャッシュ	D-cache 有効化 + コヒーレンシ処理	D-cache は 意図的に OFF (volatile+dsb でロックフリー)。その上で「ON にすると何が変わるか」を独立実験 (第6節)
同期	コア間ロック (相互排他)	ロックフリー 。full SMP を採らないので cross-core ロックが不要
対象カーネル	教育用の素の Xinu	WiFi ad-hoc メッシュ・WM・AIPL アクター・オンデバイス JIT が稼働中で 不変条件
貢献	マルチコア対応の 存在 と教材価値	何が速くなり何が速くならないか の定量的切り分け + 設計方法論

¹P. Bansal, R. Latinovich, T. Lazar, P. J. McGee, D. Brylow, “XinuPi3: Teaching Multicore Concepts Using Embedded Xinu,” CSERC ’17 (6th Computer Science Education Research Conference), Helsinki, 2017. https://epublications.marquette.edu/mscs_fac/634/

²P. J. McGee, R. Latinovich, D. Brylow, “Using Embedded Xinu and the Raspberry Pi 3 to Teach Operating Systems,” IEEE IPDPS Workshops (EduPar), 2020.

最も本質的な違いは、**対称型 SMP か、非対称ワーカープールか**である。XinuPi3 は full SMP を教育目標として採る。本研究は、既存ランタイム（アクター・ネット・WM）が非リエントラントであるという現実から出発し、full SMP は実績ある機能を全面的に作り直すリスクを負うと判断して採らない。そしてこの判断が正しいことを、二重に示した——実機実測で worker-pool が計算律速で $\times 3.99$ （ほぼ理想値）を出すこと（第5節）と、設計空間 GA が worker_pool を full_smp に対する champion として選ぶこと（第8節）である。XinuPi3 が「4 コアを使えること」を示したのに対し、本研究は「4 コアで何をどこまで使うべきか、そして何は使っても無駄か」を測る。

第二の違いは方法論にある。本研究は、(a) 実機での定量測定（計算 vs メモリ律速、プール往復の損益分岐、キャッシュ構成の支配性）、(b) D-cache 有効化という制御された比較実験、(c) AI レビューアーによる設計空間 GA と実測の照合——という、**測定と探索を軸にした接近**を採る。教育用途を主眼とする既存研究とは、問いの立て方が異なる。

3 共通設計：ワーカープール型 SMP

3.1 なぜ対称型 SMP を採らないか

素朴な発想は、ready キューを全コアで共有する対称型 SMP である。しかし既存の Xinu ランタイム（AIPL アクター、ネットワークスタック、USB、ウィンドウマネージャ）は**非リエントラント**であり、複数コアから同時に呼ばれることを想定していない。共有 ready キューを導入すれば、実機で動作実績のあるこれらの機能を全面的に作り直す必要が生じ、リスクが利得を上回る。

そこで本設計では次の非対称構成を採る。

- ・ **コア 0** は既存の単一コア OS をそのまま実行する。ready キューはコア 0 の専有であり、スケジューラ・アクター・ネットワーク・描画は一切変更しない。
- ・ **コア 1–3** は専用の計算ワーカーとして起動する。低消費電力の WFE で待機し、投入されたジョブ（仕事の範囲）を実行して完了を通知する。OS には触れず、割り込みも受けず（IRQ/FIQ マスク）、メモリ確保も行わないため、カーネルと競合し得ない。

この構成は、既存機能への退行リスクが構造的にゼロであるという点で優れている。コア 0 の挙動が変わらないからである。

3.2 ロックフリー調停が成立する理由

ワーカーとコア 0 の調停は、単一のジョブメールボックス（volatile な配列群）と dsb バリアのみで行い、スピンロックも LDREX/STREX も使わない。これが成立するのは、この移植が **D-cache を OFF にしている**ためである。すなわち全てのデータアクセスが RAM に直行するので、コア間のキャッシュコヒーレンシ問題が原理的に生じない。

これは設計上の仮定であり、実機で検証した（第 5 節）。なお D-cache を有効化すると、この仮定は崩れ、ジョブメールボックスには実際のキャッシュ管理が必要になる。

3.3 ジョブ引数は呼び出し側スタックに置く

ジョブの引数をファイルスコープの変数で渡してはならない。コア 0 はプリエンプティブであり、かつプールの利用者は複数スレッド（HTTP サーバスレッドと AIPL アクタースレッド）に及ぶ。ゆえに次の競合が起こり得る。

1. スレッド A が引数を書く。
2. A がプリエンプトされる。
3. スレッド B が引数を上書きし、プールを取って実行する。
4. A が再開し、B の引数で計算して誤った答えを返す。

本設計では API に不透明ポインタ `ud` を設け、引数を呼び出し側のスタックに置くことで、この競合を構造的に起こり得なくした。ロックで塞ぐより安全であり、副次的に性能も向上した（第 5.11 項）。

3.4 プールの排他とフォールバック

ジョブメールボックスは 1 組しかないと、プールは同時に 1 人しか使えない。既に使用中の場合、後続の呼び出し元は自分の全チャンクをその場で実行する。待たせないでデッドロックせず、並列化の利得を諦めるだけで答えは常に正しい。

同様に、ワーカーが応答しない場合はコア 0 がそのチャンクを引き取る。したがってワーカーの不調が OS を停止させることはない。

4 測定方法

4.1 時計：割り込みではなくフリーランニングカウンタを読む

当初、経過時間の測定には Xinu の `clktime/clkticks` を用いていたが、どの測定も 0ms を返した。原因はこれらがタイマ割り込みで加算される変数であり、その割り込みが安定して発火していないことにある。これはマルチコア化以前のカーネルでも同様であり、本作業が持ち込んだ問題ではない。

そこで System Timer の 1MHz フリーランニングカウンタを直接読む `clkcount()` に切り替えた。割り込みに依存しないため停止せず、分解能も 1ms から 1μs へ 1000 倍向上した。

4.2 正しさの同時検証

全てのベンチマークは、同一の仕事を逐次で 1 回、並列で 1 回実行し、両者の答えが一致するかを `agree` として毎回報告する。速いが誤った結果を、速いという理由で見逃さないためである。本レポートに載せた全測定で `agree = yes` である。

さらに、分割ロジック（剰余類による仕事の配分）は実機に投入する前に、カーネルのソースから当該関数をそのまま抜き出してホスト上で総当たり検証した。この検証により、実機に載せる前に実際に 2 件の誤りを発見・修正している（第 5.9 項）。

4.3 絶対時間ではなく比を報告する

測定中に、CPU クロックが一定でないことを発見した。再起動直後の最初の測定だけが速く、以降は安定した遅い値に落ち着く。その比は実測 2.33 で、Raspberry Pi 3 B+ の $1.4\text{GHz} \div 600\text{MHz} = 2.33$ と一致する。ベアメタル Xinu は DVFS を制御しないため、ファームウェアがクロックを変更しているものと考えられる。

したがって絶対時間は再起動からの経過に依存し、比較に使えない。一方 速度向上比は安定している。1 コア測定と 4 コア測定を同一リクエスト内で連続実行しているため、クロックの影響が相殺されるからである。実際、繰り返し測定で比は完全に一致した（例：×3.99 が 3 回とも同値）。本レポートは比を主たる指標とし、絶対時間は状態を明記した上で参考値として示す。

5 Raspberry Pi 3 (BCM2837)

5.1 対象と前提

対象は Raspberry Pi 3 B+ (BCM2837、Cortex-A53 × 4、1.4GHz) 上の Embedded Xinu (リポジトリ `xinu-raz/xinu`) である。このボードは WiFi・有線 LAN・HDMI ウィンドウマネージャ・AIPL アクターランタイム・JIT が実機で動作している一方、`include/rcu.h` に「*The arm-rpi3 port parks the secondary cores (MPIDR guard)*」とあるとおり、4 コアのうち 3 コアを起動せずに放置していた。

本移植の前提として重要なのは、このポートの MMU 設定である。`system/platforms/arm-rpi3/mmu.c` は identity map (VA = PA) を張り、MMU と I-cache は有効、D-cache は無効のままにしている。実機で読み出した SCTLR がこれを裏付ける。

```
enabled=1
sctlr=0x00c51839 M=1 C=0 I=1 Z=1
ttbr0=0x0011c000
```

Listing 1: 実機 /api/mmu の応答 (D-cache OFF の確認)

$C = 0$ 、すなわち D-cache は OFF である。よって第3節のロックフリー調停の前提が、この実機で成立していることが確認できた。

5.2 コア起動：Pi 3 固有の差分

Pi 4 / Pi 5 は AArch64 で PSCI やスピンテーブルによりコアを起動するが、Pi 3 は 32bit (arm_64bit=0) で動作しており、機構が異なる。32bit のファームウェアはコア 0 しか起動せず、コア 1-3 は armstub7 の中で自分専用のメールボックス 3 を監視して待機している。そこへ入口アドレスを書き込み、sev で起こす。

```
/* ARM-local per-core mailbox 3 SET register. The 32-bit Pi 3 firmware leaves
 * cores 1-3 spinning in armstub7 on exactly this word; writing a jump target
 * and issuing SEV releases the core to that address. This MMIO page is
 * already Device-mapped by mmu.c (0x40000000..0x40FFFFFF). */
#define ARM_LOCAL_MBOX3_SET(core) \
    ((volatile unsigned long *) (0x4000008CUL + 0x10UL * (unsigned long)(core)))
```

Listing 2: ARM ローカルのコア別メールボックス 3 (system/platforms/arm-rpi3/smp.c)

この MMIO 領域は既に mmu.c が Device 属性で写像済みであったため、追加の写像は不要であった。起こされたコアは start.S に追加した _smp_start トランポリンに入り、SVC モードへ正規化し、IRQ/FIQ をマスクし、コア専用スタックを設定して C の入口へ入る。そこでコア 0 と同一の MMU 設定 (コア 0 が構築した L1 テーブルを共有し、MMU と I-cache を有効化、D-cache は無効のまま) を適用する。設定を揃えるのは体裁の問題ではなく、4 コアの時間比較を公平にするためである。

なお、コア解放の手掛かりを 2 系統用意した。armstub7 のメールボックスと、start.S 側の smp_release[] である。後者はファームウェアが 4 コアとも _start に流し込む実装だった場合の保険である。この配列は **意図的に .bss ではなく .data に置いた**。コア 0 が .bss をゼロクリアしている最中に、待機中のコアが古い値を読むことを防ぐためである。

Table 2: 起動が正しく構成されたことの確認 (リンク結果)

シンボル	アドレス	セクション
_smp_start	0x00008280	.text
smp_release	0x0010e980	.data (D)
smp_stacktop	0x0010e990	.data (D)

実機での結果

最大の不確実性は「32bit ファームウェアが本当にこのメールボックスでコアを解放するか」であった。実機で確認された。

```
SMP bench kind=nqueens
cores_online = 4
```

Listing 3: /bench の応答 (4 コアが起動している)

仮に外れていても、起動待ちは短くバウンドしており、応答しないコアは offline 扱いで 1 コアにフォールバックして起動は継続する設計とした。

5.3 実測 1：計算律速の処理は 4 コアでスケールする

表 3 に計算律速のベンチマークを示す。値は全て安定状態 (再起動後、クロックが落ち着いた状態、アクター未起動) での測定であり、第4.3項のとおり比を主指標とする。

表 3 の N-Queens の値は、第 5.10 項に述べる測定バグの修正後のものである。修正前は nqueens n=12 が $\times 4.42(!)$ という物理的にあり得ない値を安定して示していた。n=11 と n=13 の 1 コア

Table 3: Pi 3：計算律速の速度向上 (/bench、cores_online = 4)

ベンチマーク	性質	1 コア (μs)	4 コア (μs)	速度向上	agree
dining n=5	純レジスタ演算	119,626	29,920	×3.99	yes
nqueens n=12	純計算 (再帰)	167,052	45,637	×3.66	yes
nqueens n=11	純計算 (再帰)	—	—	×3.33	yes
nqueens n=13	純計算 (再帰)	—	—	×3.12	yes
primes n=200000	計算 (下記参照)	155,686	75,958	×2.04	yes

／4 コアの絶対時間を「—」としたのは、修正後の再測がクロック状態の異なるタイミングで行われ、速度向上比 (クロックに対して安定) のみが横並びで信頼できるためである (第 4.3 項)。dining が理想値 $\times 4.00$ にほぼ届く $\times 3.99$ を示したことは、本設計の妥当性を端的に示している。この処理はメモリに一切触れない純粋なレジスタ演算であり、コア間で奪い合う資源が存在しないためである。

primes だけが $\times 2.04$ と低い。これは偶然ではなく、仕事の配分がこの問題の構造と共鳴した結果である。スライド 4 で配ると、コア 0 とコア 2 は「4 の倍数」と「 $4n+2$ 」すなわち**全て偶数**を担当し、 $d=2$ で即座に合成数と判明して一瞬で終わる。一方コア 1 と 3 が奇数を全て引き受ける。つまり実質 2 コアしか働いておらず、それがそのまま $\times 2.04$ という数字になっている。N-Queens では同じスライド配分が正しく機能している (コストが列位置に対してなめらかで、偶奇と相関しないため)。ブロック巡回配分に変えれば解消するが、これはベンチマークの均衡の問題であり、本レポートの結論には影響しない。

5.4 実測 2：メモリ律速の処理はスケールしない

表 4 に、大量のメモリ書込 (fill) の測定を示す。これは描画 (フレームバッファへの書込) と同じ性質の処理であり、ウィンドウシステムを並列化すべきかを判定するために用意した。

Table 4: Pi 3：メモリ書込律速の速度向上 (fill、8 パス)

語数	1 コア (μs)	4 コア (μs)	速度向上
64	5	13	×0.38
256	11	18	×0.61
1,024	42	37	×1.13
4,096	174	136	×1.27
16,384	689	511	×1.34
65,536	2,665	2,137	×1.24
262,144 (=1 MB)	11,007	8,125	×1.35

2 つの重要な事実が読み取れる。

第一に、**どれだけ仕事を大きくしても $\times 1.35$ 程度で頭打ちになる**。1 MB (1280×800 の画面の約 1/4 に相当) を書いても $\times 1.35$ であり、計算律速の $\times 3.99$ とは比較にならない。なお高クロック状態で測ると同じ 1 MB が $\times 0.95$ 、すなわち **4 コアの方がわずかに遅い**。いずれの状態でも、並列化に意味がないという結論は変わらない。

第二に、**小さい仕事は並列化すると遅くなる**。64 語では $\times 0.38$ 、すなわち 2.6 倍遅い。これはプールの往復コストに負けているためである。

5.5 実測 3：コア数スケーリング (Amdahl 曲線)

これまでは 4 コアのみを測ったが、/bench に cores パラメータを足し、同一の仕事を $1 \cdot 2 \cdot 3 \cdot 4$ コアで走らせて増分の効き方を測った (cores=1 が逐次基準 $\times 1.00$)。

3 つの性質が、コア数の関数として見える。

純計算はコアを足すほど線形に伸びる。dining は $\times 0.99 / \times 1.99 / \times 2.99 / \times 3.99$ と完全な直線を描く。メモリに触れない純レジスタ演算であり、コアあたり 1 の伸びが理想通り積み上がる。nqueens も $\times 3.65$ までほぼ線形で、逓減はプール往復と分割粒度による。

Table 5: Pi 3：コア数に対する速度向上 (cores=1..4、全て agree=yes)

ベンチマーク	1 コア	2 コア	3 コア	4 コア	性質
dining n=5	×0.99	×1.99	×2.99	×3.99	完全に線形 (理想)
nqueens n=12	×1.00	×1.94	×2.82	×3.65	ほぼ線形
primes n=200000	×0.99	×1.02	×2.04	×2.04	偶奇共鳴 (下記)
fill 1MB	×0.99	×0.91	×0.98	×0.93	全域でスケールせず

メモリ律速はコアを足しても平坦。fill (1MB) は 1-4 コアで $\times 0.9$ – $\times 1.0$ 付近をさまよい、どのコア数でも伸びない。第 5.4 項の「メモリ律速はスケールしない」が、単一点ではなくスケーリング曲線全体として平坦であることが確認できる。

primes の偶奇共鳴が、コア数の関数として露呈する。 $\times 1.02$ (2 コア) $\rightarrow \times 2.04$ (4 コア) という飛び方は偶然ではない。ストライド分割では、2 コアだとコア 0 が「偶数」を全て担当し、それらは $d = 2$ で即座に合成数と判明して一瞬で終わる——**実質 1 コアしか働かず** $\times 1.02$ に留まる。4 コアでも偶数を担当する 2 コアが遊ぶため $\times 2.04$ 。3 コア (ストライド 3) は剰余類が偶奇混在で均衡し $\times 2.04$ まで戻る。第 5.10 項で述べた負荷偏りが、コア数スイープで明瞭に可視化された形である。

5.6 実測 4：プールの往復コストと損益分岐点

何もしないジョブを投入して、プール自体の往復コストを測定した (null)。結果は **7–11 μ s** であった。

これが並列化の**損益分岐点**である。すなわち、これを下回る仕事をワーカーに投げると必ず損をする。効率よく利得を得るには、その 10 倍、すなわち **1 メッセージあたり 70 μ s** 以上の純計算が必要となる。この数値が、以降の適用判断の基準となった。

5.7 適用判断 1：ウィンドウシステムは並列化しない

「ウィンドウシステムなどのアプリケーションが 4 コアで速くなるか」という問いに対し、上記の測定は否と答える。理由は 2 つあり、どちらか一方だけでも致命的である。

1. **描画はメモリ律速である**。このポートは D-cache が OFF なので、描画とはフレームバッファへの書込そのものであり、第 5.4 項の fill と同じ性質を持つ。測定値は $\times 0.95$ – $\times 1.35$ であり、コアを足しても速くならない。
2. **粒度が足りない**。ウィンドウシステムの描画 1 回は小さい。たとえばカーソルは $12 \times 12 = 144$ 画素であり、往復コスト 7–11 μ s を下回る。実測でも 64 語の仕事は $\times 0.38$ と遅くなっている。並列化すれば体感はむしろ悪化する。

したがってウィンドウシステムには手を加えなかった。**速くならない上に遅くなる変更を入れな**いことも、工学上の判断である。このボードにおける法則は「**計算はスケールする、描画はスケールしない**」と要約できる。

5.8 適用判断 2：AIPL アクターは 1 箇所だけが該当する

Pi 3 の AIPL ランタイムは、アクター 1 体につき Xinu スレッド 1 本とメールボックス (リングバッファ+計数セマフォ) を持ち、スレッドがメッセージを取り出して dispatch() でメソッド本体を実行するモデルである。

損益分岐点 70 μ s に照らすと、ほとんどの **AIPL メソッドは対象外**である。bump や ping-pong は算術数命令と send で構成され、基準を 2–3 桁下回る。加えて send・print・wait といったカーネル呼び出しは、そもそもワーカーコアで実行できない。

唯一の例外がロードバランサの Worker.compute_nq である。


```

int ncores = smp_cores_online();
unsigned long t0 = clkcount();
long result = (long)abcl_nq_count_partial_smp((int)n, (int)first_col, ncores);
long elapsed = (long)((clkcount() - t0) / 1000UL); /* us -> ms */
...
enqueue(self_id, disp, "done", 3, ...); /* kernel call: stays on core 0 */

```

Listing 4: 対象となる構造 (apps/abcl_program.c)

重い計算が `abcl_nq_count_partial()` の一点に隔離され、その前後がカーネル呼び出しという理想的な構造である。 $n=13$ で 1 メッセージあたり約 100 ms の純計算であり、基準を 3 桁上回る。ここだけを 4 コア化し、カーネル呼び出しはコア 0 に残した。

分割の方法

`first_col` 1 つ分は 1 回の再帰呼び出しであるため、分割は 1 段下、すなわち 2 行目の合法な列で行う。各コアにその剰余類を配る。連続分割ではなくインターリーブとしたのは、コストが列位置に対して中央が重く左右対称であり、連続分割では中央のコアに仕事が偏るためである。

各コアは専用の盤面配列を持つ必要がある。`nq_recurse` は探索の下降中に `cols[]` を破壊的に更新するため、配列を共有するとコア同士が互いの探索を壊すからである。この配列を静的変数ではなく呼び出し側スタックに置いたのは、プールが埋まって `inline` 実行に落ちたスレッドと、プールを保持するスレッドとが、どちらもコア 0 上で同じ配列を奪い合うためである。

ホスト上での総当たり検証により、この分割の正しさとバランスを確認した。

```

REAL-SOURCE partial n=1..12 x every first_col x nc=1..4: ok
REAL-SOURCE summed partials vs known N-Queens counts: ok

balance n=13 first_col=6 nc=4 (per-core counts):  2233  2233  1802  1802
                                                    max/avg = 1.11

```

Listing 5: カーネルのソースから当該関数を抜き出したホスト検証

実機での結果

アクター経路 (`/api/loadbal/nqueens?n=12`) は正しく完走した。

```

nqueens n=12 submitted=12 task_id_range=1..12
nqueens-result done=1 received=12 expected=12 total=14200

```

Listing 6: アクター経由の N-Queens ($n=12$ の正解は 14200)

答えは正しく、また各 Worker の `busy_ms` に値が入るようになった (従来は壊れた `clkticks` を使っていたため常に 0 であった)。

5.9 発見した不具合

本作業では、実機に投入する前のホスト検証と、測定データの精査により、以下の不具合を発見・修正した。いずれも測定していなければ気付かなかったものである。

1. 素数分割の剰余類の誤り (自作、投入前に発見)。開始値の算出を誤り、剰余類がずれて一部の値を取りこぼし、かつストライド 1 のとき 1 を素数に数えていた。ホスト検証で捕捉。
2. ジョブ引数のスレッド間競合 (自作、投入前に発見)。第 3.3 項に述べた競合。アクターは Worker が 4 体、すなわちスレッド 4 本であり、実際に起こり得た。API に `ud` を追加して構造的に解消した。
3. `smp_parallel_sum` の再入不可 (自作、投入前に発見)。単一のジョブメールボックスを複数スレッドが同時に使うと破壊し合う。使用中なら呼び出し側で全チャンクを実行する方式で解消した。

4. タイマ割り込み由来の時計が常に 0 (既存)。第4.1項。本作業以前から存在した。clkcount() 直読で回避した。
5. 不公平な速度向上の測定 (自作、実機で発見)。第5.10項。1 コア基準と 4 コア測定に別関数を用いており、×4.42 という不可能な値を出していた。同一関数比較に修正した。

5.10 発見した測定バグ：不公平な基準が ×4.42 を生んだ

ベンチマークを別モジュールへ切り出して整理した直後、nqueens n=12 が ×4.42(!) を安定して示した。4 コアで ×4.00 を超えることは物理的に不可能である。agree = yes (逐次と並列の答えが一致) であり、仕事量も同一だったため、これは計算の誤りではなく測定の欠陥であった。

原因は、nqueens だけが 1 コアの基準と 4 コアの測定で別の関数を走らせていたことである。基準は逐次専用の sb_nqueens()、並列は sb_nq_par() で、両者は同じ値を計算するが別の関数であり、コンパイラのインライン展開もコード配置も異なる。並列側の実装が 1 コアあたり約 10% 効率が良く、 $4.00 \times 1.10 \approx 4.42$ で辻褄が合う。dining と primes が汚染を免れていたのは、偶然どちらも逐次・並列で同一関数を経由していたためである。

決定的な証拠は、モジュール移動という論理を 1 行も変えない操作だけで値が ×3.70 から ×4.42 へ動いたことである。比がコード配置に反応した以上、その比は並列性ではなくコード生成の差を測っていた。

修正は、全ベンチの 1 コア基準を「並列と同じ関数の stride=1」に統一することである。両辺が同一コードになり、残る変数はコア数だけになる。修正後、nqueens n=12 は ×3.66 に落ち、dining(×3.99) と primes(×2.04) は不変であった (元々公平だったため)。教訓は報告値の再現性だけでは正しさを保証しないことである。×4.42 は 5 回連続で 442/442/441/441/442 と完全再現した。安定した誤りは、ノイズよりも見つけにくい。物理的な上界 (4 コアなら ×4.00) という外部知識と突き合わせて初めて露見した。

5.11 副次的な発見：volatile グローバルの代償

第3.3項の競合対策として、ジョブ引数を volatile なファイルスコープ変数から呼び出し側スタックの構造体へ移したところ、速度向上比が ×3.04 から ×3.70 へ改善した。

理由は D-cache が OFF であることにある。volatile 変数は参照のたびに必ず実メモリを読むため、D-cache のないこの環境では内側ループの 1 反復ごとに RAM 往復が発生していた。通常の構造体メンバに変えたことでコンパイラがレジスタに載せられるようになった。

すなわち競合バグの修正が、同時に性能改善でもあった。D-cache を持たない環境では、volatile の乱用は通常環境より遥かに高くつく。

6 実験：D-cache を有効化する (Level 2)

6.1 動機：支配変数はキャッシュ構成である

第 5 節の測定で最も効いていた制約は、SMP の方式ではなく D-cache が OFF であることであった。メモリ律速がスケールしなかったのも (fill ×0.93)、volatile を外したら ×3.04 が ×3.66 に上がったのも (第 5.11 項)、動的ワークスティーリングを断念して静的分割にしたのも、すべて D-cache OFF が原因である。

そこで問いは：D-cache を ON にすると、メモリ律速の処理はスケールし始めるか。もしそうなら「ウィンドウシステムは並列化しない」という第 5 節の結論が覆る。この問いは SMP の方式とは独立であり、キャッシュ構成という別の軸に属する。

6.2 D-cache ON は「フラグを 1 にする」ではない

素朴に SCTL.R を 1 にするのは危険で、過去に一度 Pi 3 を brick させている (mmu.c 冒頭に記録あり)。安全に ON にするには、この移植に 1 行も存在しなかったキャッシュ管理を新規に書く必要があった。本実験で顕在化した罠は 4 つある。

1. リセット直後のキャッシュはゼロでなくゴミ。invalidate せずに C=1 にすると、コアはゴミを有効データと見なす。これが brick の原因である。set/way invalidate ループを C=1 の前に実行しなければならない（後では、いまキャッシュした本物のデータを捨ててしまう）。
2. Cortex-A53 は SMPEN なしでは非コヒーレント。mmu.c は RAM を shareable にマップしているが、A53 では shareable かつ cacheable なメモリでも CPUECTLR.SMPEN を立てるまでコア間でコヒーレントにならない。C=1 かつ SMPEN=0 だと、第 3 節のロックフリー・ジョブメールボックス (volatile + dsb) が静かに壊れる。volatile はコンパイラを縛るだけで、キャッシュは縛らない。
3. 二次コアの起動窓。二次コアは _smp_start の時点でまだ MMU もキャッシュも OFF であり、その状態でスタックポインタを RAM から読む。この読みはコア 0 のキャッシュをスヌープしない。ゆえにコア 0 はスタックアドレスを起こす前に clean せねばならず、さもなければ二次コアは null スタックで起きて最初の push で死ぬ。
4. mmu_disable の前提が崩れる。「キャッシュ OFF だから flush 不要」というコメントが C=1 で偽になる。kexec 前にキャッシュを落とすと、/upload がキャッシュ経由で書いた次カーネルのイメージが dirty ライン内に残り残され、stale メモリヘジャンプする。clean を追加した。

これらを cache.c (set/way・範囲・SMPEN の各操作を新規実装) と、mmu.c の DCACHE_ON 経路 (順序が命: invalidate と SMPEN を C=1 の前に) で処理した。

6.3 最大の障害: USB DMA はページ属性で直せない

D-cache ON でも DMA するハードとの共有メモリは非コヒーレントになる。そこでカーネル全体の DMA 地点を棚卸しした。結果は明快で、FB・電源・WiFi のメールボックスや SD/SDIO は全て PIO (DATA レジスタ経由) であり、コヒーレンシ問題を持たない。唯一の DMA エンジンには DWC USB であり、これが有線 LAN を担っている。

そして USBこそが構造的に直せない地点であった。DWC ドライバは呼び出し側のポインタをそのまま DMA アドレスに渡し、呼び出し側はヒープバッファ (memget は 8 バイト境界 = 隣の割り当てと 64 バイトのキャッシュラインを共有) と、あろうことかスタック上の局所変数 (smc9512.c のレジスタ操作が 4 バイトを DMA する) を渡す。64 バイトのラインを invalidate すれば、同じラインにある戻り番地や退避レジスタごと破棄する。これはページ属性では解けず、バウンズバッファとドライバ全域の 64 バイト境界化を要する。

この発見自体が結論である: D-cache を入れられないのはキャッシュ管理を書くのが面倒だからではなく、ドライバの DMA バッファの出自が構造的に許さないからである。

6.4 実験の切り分け: USB を解かずに問いに答える

問いは「D-cache でメモリ律速はスケールするか」であり、これに答えるのに USB を解く必要はない。そこで実験用ビルド変種を用意した。

```
-DDCACHE_EXPERIMENT -DDCACHE_ON -DOS_MINIMAL
```

DOS_MINIMAL でネットと WM を落とし、DCACHE_EXPERIMENT でさらに usbinit (唯一の DMA エンジン) を落とす。これで DMA する相手が一つも存在しない状態で C=1 にでき、USB の構造問題を回避したまま本題を測れる。ただし HDMI は残した——ユーザ要求であり、またフレームバッファの扱いが D-cache ON 実装の良い試金石でもあるためである。

HDMI を残すには 2 つの手当てが要った。FB のメールボックス要求構造体は元はスタック上にあり (GPU が stale RAM を読み address=0 = SYSERR = 画面出ず)、64 バイト境界・64 バイト詰め専用 static に移して handshake の前後で clean/invalidate した。FB のピクセル領域自体は、GPU がアドレスを返した後に mmu_set_range_noncached() で Normal Non-cacheable へ貼り替えた (アドレスは mmu_init の後に判明するため、静的な表には焼けない)。

6.5 測定ハーネス

実験変種はネットが無いので /bench を提供できない。そこでワークロード本体を apps/smpbench.c に切り出し、**HTTP 経路（既定カーネル）と起動時コンソール経路（実験カーネル）が同一のコードを呼ぶようにした**。キャッシュ ON/OFF の比較は、両者が同一実装を走らせて初めて意味を持つ——別実装なら「キャッシュ設定の差」ではなく「実装の差」を測ってしまう。この切り出しの過程で、第 5.10 項の測定バグを発見・修正している。

実験カーネルは起動時に smpbench_run_all() を実行し、fill のサイズ掃引を中心とした結果をシリアルへ出力する。

6.6 実測結果

D-cache ON カーネルを実機に投入し、起動時ハーネスの出力をシリアルで取得した。まず 3 つの前提がすべて確認された。

- **起動した (brick 回避)**。Xinu のバナーからシェルまで正常に立ち上がった。set/way invalidate を C=1 の前に置く順序が効いている。
- **cores_online = 4、全ベンチ agree = ok**。SMPEN を立てたことで、C=1 下でもロックフリー・ジョブメールボックスのコヒーレンシが保たれている。逐次と並列の答えは全ベンチで一致した。
- **sctlr = 0x00c5183d**。D-cache OFF の 0x00c51839 に対しビット 2 (C) が立ち、実機で D-cache が有効化されたことを裏付ける。

本命：メモリ律速はキャッシュでスケールする——ただし容量まで

Table 6: fill の速度向上：D-cache OFF vs ON (8 パス、作業セット掃引)

語数（作業セット）	OFF の速度向上	ON の速度向上	
64	×0.38	×2.00	
256	×0.61	×2.50	
1,024 (4 KB)	×1.13	×3.60	
4,096 (16 KB)	×1.27	×3.88	
16,384 (64 KB)	×1.34	×4.01	← 完全スケール
65,536 (256 KB)	×1.24	×2.51	← 低下開始
262,144 (1 MB)	×1.35	×1.40	← 帯域律速へ回帰

表 6 が本実験の答えである。第 5 節の結論「メモリ律速はスケールしない」は、**D-cache OFF に固有の現象**であった。D-cache ON では、作業セットがキャッシュに収まる限りメモリ書込は理想的にスケールする（64 KB で ×4.01）。

しかしスケールするのは容量までである。作業セットが 256 KB を超えると速度向上は崩れ、1 MB では ×1.40 まで戻る。これは**キャッシュ容量効果**である。小さい fill は同じ領域を 8 回書くため 2 回目以降はキャッシュに当たり、実質計算律速になってスケールする。作業セットがキャッシュ（Pi 3 の Cortex-A53 は L1 データ 32 KB / コア + 共有 L2 512 KB）を超えると、書き戻しと充填で再び**帯域律速**に戻る。速度向上が 64 KB で頂点、256 KB で崩れ始めるのは、この容量境界と整合する。

ウィンドウシステムの結論は覆るか——覆らない

一見、これは「WM は並列化しない」（第 5 節）を覆すように見える。だが覆らない。理由が 2 つある。

第一に、**実フレームバッファはキャッシュ不可にマップしてある**（第 6.3 項、mmu_set_range_noncached）。GPU がスヌープしないため、これは D-cache ON でも変えられない。表 6 の fill は**通常のキャッシュ可能バッファ**での測定であり、実描画はこの恩恵を受けない。

第二に、恩恵を受けるのは作業セットがキャッシュに収まる場合だけだが、全画面ブリットは $1280 \times 800 \times 4 = 3.9 \text{ MB}$ でキャッシュを大きく超え、仮にキャッシュ可能でも $\times 1.40$ 程度に留まる。

したがって結論は次のように精緻化される：メモリ律速の処理は、D-cache ON かつ作業セットがキャッシュ内なら 4 コアでスケールする。だが実描画は（キャッシュ不可の FB へ、キャッシュを超える量を書くため）どちらの条件も満たさず、依然スケールしない。中間バッファ上での合成のように、キャッシュ内で完結する処理を挟めば話は別である。

副産物：D-cache は単一コア性能も大きく上げる

計算律速側では、 $\text{dining}(\times 3.99) \cdot \text{primes}(\times 2.05)$ の速度向上比は D-cache OFF とほぼ不変であった（元々スケールしていたため）。一方で絶対性能は激変した。nqueens $n=12$ の 1 コア時間は D-cache OFF の約 $167,000 \mu\text{s}$ から約 $19,000 \mu\text{s}$ へ、実に 8.8 倍高速化している。クロック差（起動直後 2.33 倍）だけでは説明できず、再帰探索が触る盤面配列 `cols[]` とスタックがキャッシュに載った効果である。並列スケーリングとは独立に、D-cache は計算律速の絶対性能そのものを底上げする。

ただし本番投入はできない

以上は実験変種（DCACHE_EXPERIMENT、USB を落とした構成）での結果である。本番カーネルに D-cache を入れるには、第 6.3 項の USB DMA 問題——ヒープとスタックのバッファへのバウンス、およびドライバ全域の 64 バイト境界化——を解く必要がある。それは本実験の範囲外であり、次段階の課題である。

6.7 付録：実機シリアル出力（D-cache ON）

参考のため、実機（コールドブート）から取得した起動時ハーネスの生出力を示す。

```
===== smpbench =====
cores_online = 4
D-cache      = ON   (SCTLR.C=1, SMPEN set)
sctlr        = 0x00c5183d

-- compute-bound (expect x3.2~x4.0) --
bench          1-core us  N-core us  speedup
dining n=5      51290      12828    x3.99  ok
nqueens n=11    3716       1027     x3.61  ok
nqueens n=12    19024      4865     x3.91  ok
primes 200k     66761     32550     x2.05  ok

-- memory-store-bound: THE QUESTION (x1.0 = no benefit) --
words          1-core us  N-core us  speedup
fill 64         2         1    x2.00  ok
fill 256        5         2    x2.50  ok
fill 1024       18         5    x3.60  ok
fill 4096       70        18    x3.88  ok
fill 16384      285        71    x4.01  ok
fill 65536     1177        468    x2.51  ok
fill 262144    4682       3330    x1.40  ok

-- pool round trip (the parallelise/don't break-even) --
null           0          1    x0.00  ok

===== end smpbench =====
```

6.8 本番化の試みと限界：USB DMA は越えられなかった

第 6.3 項で「USB は構造的に難しい」と述べたが、実際に本番化を試みた。結果は部分成功であり、その失敗の様相そのものが価値ある結果なので記録する。

方式：全 USB 転送を非キャッシュ・バウンス経由に

USB DMA の障害は、バッファがヒープ（memget は 8 バイト境界＝ 64 バイトのキャッシュラインを隣と共有）やスタック上にあり、per-transfer の clean/invalidate が隣接データを巻き込むことだった。そこで per-transfer 管理を避け、全転送を専用の非キャッシュ・アリーナ経由にバウンスする方式を採った。DWC ドライバは元々、非境界バッファ用のバウンス経路（aligned_bufs + TX/RX コピー）を持つので、その切替を「D-cache ON なら常にバウンス」に変え、アリーナを非キャッシュにマップした。非キャッシュ領域は CPU も DMA も同一 RAM を直接見るので、原理上キャッシュ管理命令が一切不要になる。

実機結果：起動と HDMI は成功、USB は全滅

Table 7: D-cache 本番化（フルカーネル）の実機結果

項目	結果
D-cache ON で起動	✓ brick なし（デスクトップ表示）
HDMI / フレームバッファ	✓ 動作（FB の非キャッシュ再マップが有効）
set/way invalidate + SMPEN	✓ 機能
単一コア性能	✓ 8.8 倍（実験変種で実測）
USB（マウス・キーボード・ethernet）	× 全滅

HDMI が正しく映ることは、非キャッシュ化の機構そのもの（mmu_set_range_noncached）が正しく動作している証拠である（フレームバッファも同じ関数で非キャッシュ化しており、壊れていれば画面が乱れる）。にもかかわらず USB は全滅した。Raspberry Pi 3 ではマウス・キーボード・ethernet がすべて LAN9514 ハブの背後にあるため、USB が一つ壊れると全滅する。実機シリアルログには [webactor] autostart: ETH0 never came up が記録された。

四つの修正を試みた——いずれも解決せず

原因を、実機を最小限しか使わずオフラインのコードレビューで追った。

1. **DSB バリア追加**。非キャッシュ書き込みもライトバッファに滞留し得るため、DMA 起動前に DSB で RAM へ排出。必要だが不十分——単独では USB は復活しなかった。
2. **packet_count オーバーフロー修正**。packet_count は 10 ビット（最大 1023）で、バウンスアリーナを 128 KB に拡大した際に、小さい max_packet_size で桁溢れし得ることを発見。しかし赤鯉だった——スプリット転送では transfer.size が max_packet_size に事前制限され packet_count は常に 1 で溢れない。撤回した。
3. **スプリット転送ロジックの精査**。Pi 3 は全転送がスプリット（ハブ背後）なので、バウンスとスプリット再試行の相互作用を徹底トレースした。ロジックは正しかった——dma_address の保持、再アーム、コピーバックのオフセットはすべて整合していた。
4. **アリーナ非キャッシュ化の SMP タイミング修正**。最有力仮説。非キャッシュ化を実行時（usbinit 内）に、しかも現在コアだけ TLB 無効化で行っていたが、SMP のワーカーコアはそれ以前にアリーナがキャッシュ可能なまま起動しており、コア間でマッピングが不整合だった。アリーナのアドレスはコンパイル時に確定するので、mmu_init で全コア起動前・D-cache 有効化前に非キャッシュをベイクするよう変更した。しかしこれも USB を復活させなかった。

結論：本番化は USB DMA に阻まれた

四つの修正を尽くしても USB は動かなかった。バウンスロジックとスプリット処理は精査済みで正しく、コヒーレンシ機構も（HDMI が証明する通り）機能している。それでも USB が全滅する——真因は、これらより深い、実機レベルの精密診断（USB 列挙の詳細なシリアルログ等）

を要する相互作用にある。手元のシリアルアダプタ（Prolific、macOS で不安定）ではそこまで追えなかった。

したがって本レポートの範囲では、**D-cache** は計算と表示には本番投入できるが、**USB DMA** が最後の、そして越えられない壁であると結論する。第 6.3 項の構造的予測——USB のバッファ出自が本質的な障害——が、バウンスという迂回策すら退ける形で、より強く裏付けられた。8.8 倍の単一コア高速化という果実は実在するが、それを本番で収穫するには、USB ドライバの DMA 経路をより根本的に作り直す必要がある。これは明確に次段階の課題である。

7 ボード横断比較

Table 8: ボード別ワーカープール型 SMP の比較（rpi4 / rpi5 は追記予定）

	Pi 3 (BCM2837)	Pi 4 (BCM2711)	Pi 5 (BCM2712)	備考
CPU	Cortex-A53 ×4	Cortex-A72 ×4	Cortex-A76 ×4	
実行状態	AArch32 (32bit)	AArch64	AArch64	Pi 3 のみ 32bit
コア解放	armstub7 mailbox 3	spin table	PSCI	第5.2項
D-cache	OFF	OFF	OFF	ロックフリーの前提
最大速度向上	×3.99	—	—	計算律速
プール往復	7–11 μs	—	—	並列化の損益分岐

Raspberry Pi 4 および 5 についても、本レポートと同一の枠組み（同一の /bench ルート、同一の agree 検証、同一の比による報告）で測定し、本節の表と各ボード節に追記する予定である。

8 設計空間 GA：予測 vs 実機実測

8.1 狙い：GA の予測を実測と突き合わせる

本レポートのここまでは実機実測である。一方、この一連の作業には別の系譜がある——aice-pi-evolution の設計空間 GA（.aice → Gemini レビューアー → MAP-Elites → champion）。カーネル設計を 8 軸の遺伝子（scheduler / memory_alloc / ipc_primitive / isolation / arch_target / power_mgmt / concurrency_safety / smp_model）で表し、Gemini レビューアー 4 本で採点して世代更新する。

この Round の独自性は、レビューアー rubric を本セッションの実機実測に基づいて書いたことである。ゆえに GA の champion を「実測で分かっている事実」と直接突き合わせられる——設計空間 GA の予測 vs 実機実測。

8.2 結果：worker_pool が champion（実測が裏付ける）

seed=4 / gen=10、14 個体。MAP-Elites のセル軸は smp_model。

Table 9: GA champion（MAP-Elites セル別ベスト）

smp_model	best score	n	内訳
worker_pool	0.622 (champion)	9	conc=lock_free
full_smp	0.397	5	conc=lock_free
single_core	(出現せず)	0	—

worker_pool (0.622) が full_smp (0.397) に勝った。これは Pi 5 の GA Round (0.507 vs 0.371) に続く再現であり、かつ Pi 3 では実機実測がこれを裏付けた——worker_pool を実装して dining ×3.99・nqueens ×3.66 を実証している。GA の見出し（「worker_pool を磨け、full_smp に走るな」）は、実測の前に、そして独立に、正しい方向を示した。

8.3 一致とアーティファクト

champion の 8 軸を実測と照合すると、明確な線が引ける。

Table 10: GA 予測 vs 実機実測

軸	GA champion	実機実測	判定
smp_model	worker_pool	×3.99 実証	✓ 一致
concurrency_safety	lock_free (12/14)	volatile+dsb で成立	✓ 一致
arch_target	arm_rpi3 (12/14)	BCM2837 A53	✓ 一致
isolation	mmu_per_process (14/14)	none/user_kernel_split が安全	× アーティファクト
ipc_primitive	capability/tuple_space	sem_mailbox (AIPL 保護)	× アーティファクト

主軸 (smp_model・concurrency_safety・arch) は実測と一致する一方、副軸 (isolation・ipc) はアーティファクトである。全 14 個体が isolation=mmu_per_process に固着したのはシード増殖であり、rubric が減点しても消えなかった。ipc=capability/tuple_space は実測では AIPL/JIT/メッシュを全減させる選択だが champion に残った。これは Pi 5 の kernel evolution でも見た「小規模 GA は方向性まで信頼、全遺伝子詳細はアーティファクト混入」パターンの再現である。

8.4 最大の発見：支配変数が schema に無い

本セッションの実測で最も効いた変数は **D-cache** (単一コア ×8.8、メモリ律速のスケール可否を決める、第 6 節) だった。しかし 8 軸の遺伝子 schema にキャッシュ構成の軸は存在しない。D-cache は memory_alloc / power_mgmt を通じて間接的にしか表現されず、**GA は経験的に最重要と判明した変数を構造的に探索できない**。

したがって具体的な提言が出る：schema に cache_policy 軸(dcache_off / dcache_on_bounce_usb / dcache_on_no_dma)を追加すべきである。そうすれば GA が D-cache のトレードオフ (×8.8 vs USB DMA の壁) を直接探索できる。

8.5 結論：GA は方向を、実測は真実を語る

- **価値**：GA は「worker_pool を磨け」という方向を、実測の前に独立に正しく示した。Pi 3 / Pi 5 で再現し、Pi 3 では実測が裏付けた。
- **限界 1**：全遺伝子の詳細 (isolation・ipc) はアーティファクト混入。主軸のみ信頼でき、副軸は人手で実測に合わせて書き直すべき。
- **限界 2**：経験的な支配変数 (D-cache) が schema に無ければ、GA はそれを見つけられない。実機実測が GA を補完する。

9 結論

1. 遊休していた 3 コアを、既存 OS を一切変更せずに計算へ動員できた。計算律速の処理で最大 ×3.99 (理想値 ×4.00) を実測した。
2. **何でも速くなるわけではない**。メモリ書込律速の処理は ×0.95–×1.35 に留まる。またプールの往復コスト 7–11 μs を下回る小さな仕事は、並列化すると**遅くなる** (実測 ×0.38)。この 2 つの境界が適用対象を決める。
3. 上記の測定に基づき、ウィンドウシステムは並列化しないという判断を下した。描画はメモリ律速であり、かつ 1 回の描画が往復コストを下回るため、速くならないばかりか遅くなるからである。速くならない変更を入れないことも、工学上の成果である。
4. AIPL アクターについては、判定基準 (1 メッセージあたり 70 μs 以上の純計算を持ち、send/print/wait を含まない) を満たす唯一の対象に適用した。

5. 測定基盤そのものにも 3 つの落とし穴があった。タイマ割り込みに依存した時計が常に 0 を返していたこと（第4.1項）、CPU クロックが一定でないこと（第4.3項）、そして 1 コア基準と並列で別関数を測って $\times 4.42$ という不可能な値を出していたこと（第5.10項）である。いずれも、比を指標とし、フリーランニングカウンタを直読し、両辺で同一関数を測ることで解決した。安定して再現する誤りは、ノイズより見つけにくい。
6. 支配変数はキャッシュ構成であった（第 6 節）。D-cache を有効化すると、メモリ律速の処理は作業セットがキャッシュに収まる限り理想的にスケールし（ $\times 4.01$ ）、超えると帯域律速へ回帰する（ $\times 1.40$ ）。「メモリ律速はスケールしない」は D-cache OFF に固有の現象だった。ただし実描画は（キャッシュ不可の FB へキャッシュを超える量を書くため）依然スケールせず、ウィンドウシステムの結論は精緻化されつつも覆らない。D-cache 有効化には set/way invalidate・SMPEN・DMA コヒーレンシが要り、USB がスタック上のバッファを DMA する構造が最大の障害である——これは本番投入に残された次段階の課題である。